

Project 3: Probabilistic Optimization

Layout Optimization

- 0.1 Layout for $n = 3$
- 0.2 Layout for $n = 5$
- 0.3 Layout for $n = 7$
- 0.4 Plot minimum distance against number of turbines in optimized layouts for all cases $n \in [2, 10]$
- 0.5 Plot minimum separation distance against efficiency using the theoretical efficiency formula
- 0.6 Plot minimum separation distance against efficiency using the theoretical efficiency formula

1 Gaussian Process Fitting

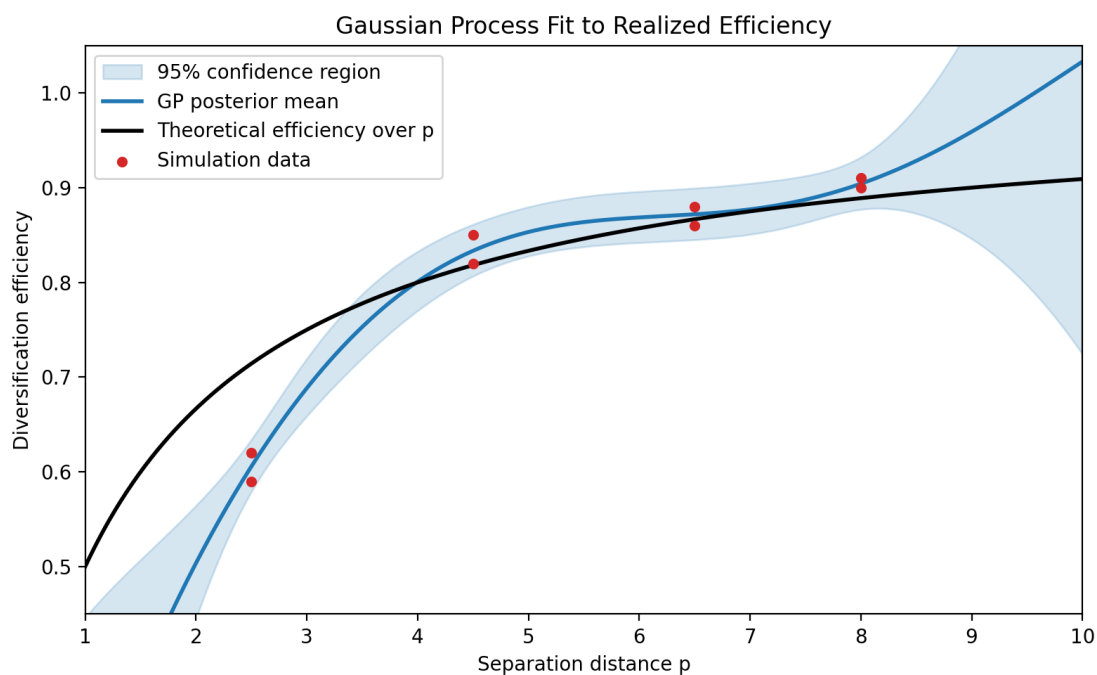


Figure 1: Gaussian process fit for realized diversification efficiency.

Code

The implementation is included in Appendix A.2.

A Code

A.1 Portfolio Layout Optimization

```
1 import numpy as np
2 import os
3 from pathlib import Path
4
5 os.environ.setdefault("JAX_PLATFORMS", "cpu")
6 os.environ.setdefault("JAX_PLATFORM_NAME", "cpu")
7 os.environ.setdefault("JAX_ENABLE_X64", "True")
8 os.environ.setdefault("MPLCONFIGDIR", str(Path(__file__).with_name(".
    matplotlib")))
9
10 import matplotlib
11 matplotlib.use("Agg")
12 import matplotlib.pyplot as plt
13 from matplotlib.patches import Circle
14
15 import jax
16 from jax import config
17 config.update("jax_enable_x64", True)
18
19 import jax.numpy as jnp
20 import optimistix as optx
21 from slsqp_jax import SLSQP, SLSQPConfig, ToleranceConfig
22
23 ZONE_1 = np.array([5.0, 5.0])
24 INNER_RADIUS = 1.5
25 OUTER_RADIUS = 4.0
26 ZONE_2 = np.array([7.5, 2.5])
27 ZONE_3 = np.array([2.5, 7.5])
28 EXCLUSION_RADIUS = 1.2
29 LOG_FILE = Path(__file__).with_name("portfolio_layout_optimization.log")
30
31 def log(message):
32     print(message, flush=True)
33     with LOG_FILE.open("a") as f:
34         f.write(message + "\n")
35
36 def feasible_points(points):
37     d_zone_1 = np.linalg.norm(points - ZONE_1, axis=1)
38     d_zone_2 = np.linalg.norm(points - ZONE_2, axis=1)
39     d_zone_3 = np.linalg.norm(points - ZONE_3, axis=1)
40
41     return (
42         (d_zone_1 >= INNER_RADIUS)
43         & (d_zone_1 <= OUTER_RADIUS)
44         & (d_zone_2 >= EXCLUSION_RADIUS)
45         & (d_zone_3 >= EXCLUSION_RADIUS)
46     )
47
48 def sample_feasible_points(rng, count):
49     points = []
```

```
50 while len(points) < count:
51     candidates = rng.uniform(1.0, 9.0, size=(4 * count, 2))
52     valid = candidates[feasible_points(candidates)]
53     points.extend(valid[: count - len(points)])
54
55     return np.array(points)
56
57 def random_feasible_layout(rng, n):
58     return sample_feasible_points(rng, n).reshape(2 * n)
59
60 def project_point(point, n_passes=5):
61     point = point.copy()
62     centers = (ZONE_2, ZONE_3)
63
64     for _ in range(n_passes):
65         direction = point - ZONE_1
66         distance = np.linalg.norm(direction)
67         if distance < 1e-12:
68             direction = np.array([1.0, 0.0])
69             distance = 1.0
70
71         if distance < INNER_RADIUS:
72             point = ZONE_1 + direction / distance * INNER_RADIUS
73         elif distance > OUTER_RADIUS:
74             point = ZONE_1 + direction / distance * OUTER_RADIUS
75
76         for center in centers:
77             direction = point - center
78             distance = np.linalg.norm(direction)
79             if distance < 1e-12:
80                 direction = np.array([1.0, 0.0])
81                 distance = 1.0
82             if distance < EXCLUSION_RADIUS:
83                 point = center + direction / distance * EXCLUSION_RADIUS
84
85     return point
86
87 def project_layout(x, n):
88     points = x.reshape(n, 2).copy()
89     valid = feasible_points(points)
90     for i in np.where(~valid)[0]:
91         points[i] = project_point(points[i])
92     return points.reshape(2 * n)
93
94 def repair_layout(x, n, rng):
95     points = x.reshape(n, 2).copy()
96     valid = feasible_points(points)
97     if not np.all(valid):
98         points[~valid] = sample_feasible_points(rng, np.sum(~valid))
99     return points.reshape(2 * n)
100
101 def min_pairwise_distance(x, n=2):
102     pts = x.reshape(n, 2)
103     best = np.inf
```

```
104
105     for i in range(n):
106         for j in range(i + 1, n):
107             best = min(best, np.linalg.norm(pts[i] - pts[j]))
108
109     return best
110
111 def layout_score(x, n):
112     points = x.reshape(n, 2)
113     if not np.all(feasible_points(points)):
114         return -np.inf
115     return min_pairwise_distance(x, n)
116
117 def jax_ineq_constraints(z, args):
118     n = args
119     points = z[:-1].reshape((n, 2))
120     p = z[-1]
121
122     d1 = jnp.linalg.norm(points - jnp.array(ZONE_1), axis=1)
123     d2 = jnp.linalg.norm(points - jnp.array(ZONE_2), axis=1)
124     d3 = jnp.linalg.norm(points - jnp.array(ZONE_3), axis=1)
125
126     constraints = [
127         d1 - INNER_RADIUS,
128         OUTER_RADIUS - d1,
129         d2 - EXCLUSION_RADIUS,
130         d3 - EXCLUSION_RADIUS,
131     ]
132
133     for i in range(n):
134         for j in range(i + 1, n):
135             constraints.append(jnp.array([jnp.linalg.norm(points[i] -
136                 points[j]) - p]))
137
138     return jnp.concatenate(constraints)
139
140 def jax_obj(z, args):
141     return -z[-1], None
142
143 def refine_layout_jax(x0, n, max_steps=100):
144     z = jnp.array(np.concatenate([x0.copy(), [min_pairwise_distance(x0, n)
145         ]]))
146     bounds = jnp.array([[1.0, 9.0]] * (2 * n) + [[0.0, 8.0]])
147     n_ineq = 4 * n + n * (n - 1) // 2
148
149     solver = SLSQP(
150         ineq_constraint_fn=jax_ineq_constraints,
151         n_ineq_constraints=n_ineq,
152         bounds=bounds,
153         config=SLSQPConfig(tolerance=ToleranceConfig(rtol=1e-8, atol=1e-8,
154             max_steps=max_steps)),
155     )
156
157     try:
```

```

155     sol = optx.minimise(jax_obj, solver, z, args=n, has_aux=True,
156                        max_steps=max_steps)
157 except Exception:
158     if layout_score(x0, n) == -np.inf:
159         return x0, -np.inf
160     return x0, min_pairwise_distance(x0, n)
161
162 x = np.array(sol.value[:-1])
163 score = layout_score(x, n)
164 if score == -np.inf:
165     if layout_score(x0, n) == -np.inf:
166         return x0, -np.inf
167     return x0, min_pairwise_distance(x0, n)
168
169 return x, score
170
171 def outer_circle_layout(n, phase):
172     angles = phase + np.arange(n) * 2.0 * np.pi / n
173     points = ZONE_1 + OUTER_RADIUS * np.column_stack([np.cos(angles), np.
174                                                         sin(angles)])
175     return points.reshape(2 * n)
176
177 def circle_points(center, radius, count):
178     angles = np.linspace(0.0, 2.0 * np.pi, count, endpoint=False)
179     return center + radius * np.column_stack([np.cos(angles), np.sin(
180                                                 angles)])
181
182 def annulus_ring_points(n_rings=6, points_per_ring=240):
183     radii = np.linspace(INNER_RADIUS, OUTER_RADIUS, n_rings + 2)[1:-1]
184     return np.vstack([circle_points(ZONE_1, radius, points_per_ring) for
185                        radius in radii])
186
187 def boundary_candidate_points(rng, points_per_circle=1000, interior_count
188                               =5000, n_rings=6, points_per_ring=240):
189     candidates = [
190         circle_points(ZONE_1, OUTER_RADIUS, points_per_circle),
191         circle_points(ZONE_1, INNER_RADIUS, points_per_circle),
192         annulus_ring_points(n_rings, points_per_ring),
193         circle_points(ZONE_2, EXCLUSION_RADIUS, points_per_circle),
194         circle_points(ZONE_3, EXCLUSION_RADIUS, points_per_circle),
195         sample_feasible_points(rng, interior_count),
196     ]
197
198     candidates = np.vstack(candidates)
199     candidates = candidates[feasible_points(candidates)]
200     return candidates
201
202 def greedy_maxmin_layout(candidates, n, start_idx):
203     selected = [candidates[start_idx]]
204     available = np.ones(len(candidates), dtype=bool)
205     available[start_idx] = False
206     nearest_dist = np.linalg.norm(candidates - selected[0], axis=1)
207
208     while len(selected) < n:

```

```
204     masked_dist = np.where(available, nearest_dist, -np.inf)
205     next_idx = int(np.argmax(masked_dist))
206     selected.append(candidates[next_idx])
207     available[next_idx] = False
208     new_dist = np.linalg.norm(candidates - candidates[next_idx], axis
209                               =1)
210     nearest_dist = np.minimum(nearest_dist, new_dist)
211
212     return np.array(selected).reshape(2 * n)
213
214 def improve_layout_by_swaps(layout, candidates, n, max_passes=4,
215                             trial_count=300):
216     selected = layout.reshape(n, 2).copy()
217     best_score = min_pairwise_distance(selected.reshape(2 * n), n)
218
219     for _ in range(max_passes):
220         improved = False
221         for i in range(n):
222             others = np.delete(selected, i, axis=0)
223             distances = np.linalg.norm(candidates[:, None, :] - others[
224                                     None, :, :], axis=2)
225             candidate_scores = np.min(distances, axis=1)
226             trial_idx = np.argsort(candidate_scores)[-trial_count:]
227
228             for idx in trial_idx:
229                 trial = selected.copy()
230                 trial[i] = candidates[idx]
231                 score = min_pairwise_distance(trial.reshape(2 * n), n)
232                 if score > best_score:
233                     selected = trial
234                     best_score = score
235                     improved = True
236
237         if not improved:
238             break
239
240     return selected.reshape(2 * n)
241
242 def greedy_candidate_layouts(rng, n, n_starts=8, use_swaps=True):
243     candidates = boundary_candidate_points(rng)
244     if len(candidates) < n:
245         return []
246
247     anchors = [
248         int(np.argmax(candidates[:, 0])),
249         int(np.argmin(candidates[:, 0])),
250         int(np.argmax(candidates[:, 1])),
251         int(np.argmin(candidates[:, 1])),
252     ]
253     if n_starts > len(anchors):
254         anchors.extend(rng.choice(len(candidates), size=n_starts - len(
255             anchors), replace=False))
256
257     layouts = []
```

```

254     for start_idx in anchors[:n_starts]:
255         layout = greedy_maxmin_layout(candidates, n, int(start_idx))
256         if use_swaps:
257             layout = improve_layout_by_swaps(layout, candidates, n)
258         if np.all(feasible_points(layout.reshape(n, 2))):
259             layouts.append(layout)
260
261     return layouts
262
263 def diverse_candidate_subset(candidates, max_candidates=450):
264     if len(candidates) <= max_candidates:
265         return candidates
266
267     anchors = [
268         int(np.argmax(candidates[:, 0])),
269         int(np.argmin(candidates[:, 0])),
270         int(np.argmax(candidates[:, 1])),
271         int(np.argmin(candidates[:, 1])),
272     ]
273     selected = []
274     used = np.zeros(len(candidates), dtype=bool)
275     nearest = np.full(len(candidates), np.inf)
276
277     for idx in anchors:
278         if not used[idx]:
279             selected.append(idx)
280             used[idx] = True
281             nearest = np.minimum(nearest, np.linalg.norm(candidates -
282                 candidates[idx], axis=1))
283
284     while len(selected) < max_candidates:
285         scores = np.where(used, -np.inf, nearest)
286         idx = int(np.argmax(scores))
287         selected.append(idx)
288         used[idx] = True
289         nearest = np.minimum(nearest, np.linalg.norm(candidates -
290             candidates[idx], axis=1))
291
292     return candidates[selected]
293
294 def find_clique(adjacency, n, verbose=False):
295     degrees = np.array([len(neighbors) for neighbors in adjacency])
296     calls = 0
297
298     def expand(clique, candidates):
299         nonlocal calls
300         calls += 1
301         if verbose and calls % 10000 == 0:
302             log(
303                 f"░░░░░clique░search░calls={calls},░depth={len(clique)},░
304                     candidates={len(candidates)}",
305             )
306         if len(clique) == n:
307             return clique

```

```

305     if len(clique) + len(candidates) < n:
306         return None
307
308     ordered = sorted(candidates, key=lambda idx: degrees[idx], reverse
309                     =True)
310     for idx in ordered:
311         next_candidates = candidates.intersection(adjacency[idx])
312         result = expand(clique + [idx], next_candidates)
313         if result is not None:
314             return result
315         candidates.remove(idx)
316         if len(clique) + len(candidates) < n:
317             return None
318     return None
319
320 result = expand([], set(range(len(adjacency))))
321 if verbose:
322     status = "found" if result is not None else "not_found"
323     log(f"UUUUclique_search_{status},calls={calls}")
324 return result
325
326 def clique_layout_for_distance(candidates, n, distance, verbose=False):
327     diff = candidates[:, None, :] - candidates[None, :, :]
328     far_enough = np.linalg.norm(diff, axis=2) >= distance
329     adjacency = [set(np.flatnonzero(far_enough[i])) - {i} for i in range(
330                 len(candidates))]
331     if verbose:
332         avg_degree = np.mean([len(neighbors) for neighbors in adjacency])
333         log(f"UUtrying_clique_distance={distance:.6f},candidates={len(
334             candidates)},avg_degree={avg_degree:.1f}")
335     clique = find_clique(adjacency, n, verbose)
336     if clique is None:
337         return None
338     return candidates[clique].reshape(2 * n)
339
340 def discrete_clique_layout(rng, n, max_candidates=400, n_search=16,
341 verbose=False):
342     raw_candidates = boundary_candidate_points(rng)
343     candidates = diverse_candidate_subset(raw_candidates, max_candidates)
344     if verbose:
345         log(
346             f"n={n}:discrete_clique_candidates_{len(candidates)}/{len(
347                 raw_candidates)},"
348             f"binary_search_steps={n_search}",
349         )
350     low = 0.0
351     high = 8.0
352     best_layout = None
353     best_score = -np.inf
354
355     for step in range(n_search):
356         mid = 0.5 * (low + high)
357         layout = clique_layout_for_distance(candidates, n, mid, verbose)
358         if layout is None:

```



```

354         high = mid
355         if verbose:
356             log(f"n={n}: clique_step_{step+1}/{n_search} infeasible at_{mid:.6f}")
357     else:
358         low = mid
359         best_layout = layout
360         best_score = layout_score(layout, n)
361         if verbose:
362             log(f"n={n}: clique_step_{step+1}/{n_search} feasible at_{mid:.6f}, score={best_score:.6f}")
363
364     return best_layout, best_score
365
366 def cross_entropy(f, n, k_max, m=100, m_elite=10, rng=None):
367     """
368     f: objective function
369     P: proposal distribution
370     m: sample size
371     m_elite: number of samples to use when refitting
372     """
373     # if not 0 < m_elite <= m:
374     #     raise ValueError("m_elite must be between 1 and m")
375     # rng = np.random.default_rng() if rng is None else rng
376
377     dim = 2 * n      # beta, HML coords
378
379     initial_samples = np.array([random_feasible_layout(rng, n) for _ in
380                                range(m)])
381     initial_scores = np.array([f(sample) for sample in initial_samples])
382     elite_samples = initial_samples[np.argsort(initial_scores)[-m_elite:]]
383
384     mu = np.mean(elite_samples, axis=0)
385     sigma = np.cov(elite_samples, rowvar=False) + 1e-4 * np.eye(dim)
386
387     best_x = None
388     best_score = -np.inf
389
390     for _ in range(k_max):
391         samples = rng.multivariate_normal(mu, sigma, m)
392         samples = np.array([repair_layout(sample, n, rng) for sample in
393                             samples])
394         scores = np.array([f(sample) for sample in samples])
395
396         i_best = np.argmax(scores)
397         if scores[i_best] > best_score:
398             best_score = scores[i_best]
399             best_x = samples[i_best].copy()
400
401         elite_idx = np.argsort(scores)[-m_elite:]
402         elite_samples = samples[elite_idx]
403
404         mu = np.mean(elite_samples, axis=0)
405         sigma = np.cov(elite_samples, rowvar=False) + 1e-4 * np.eye(dim)

```

```
404     return best_x, best_score
405
406
407 def cma_es(f, n, k_max, m=100, m_elite=None, rng=None, sigma0=2.0):
408     dim = 2 * n
409     m_elite = m // 2 if m_elite is None else m_elite
410
411     weights = np.log(m_elite + 0.5) - np.log(np.arange(1, m_elite + 1))
412     weights = weights / np.sum(weights)
413     mu_eff = 1.0 / np.sum(weights ** 2)
414
415     c_sigma = (mu_eff + 2.0) / (dim + mu_eff + 5.0)
416     d_sigma = 1.0 + 2.0 * max(0.0, np.sqrt((mu_eff - 1.0) / (dim + 1.0)) -
417                               1.0) + c_sigma
418     c_c = (4.0 + mu_eff / dim) / (dim + 4.0 + 2.0 * mu_eff / dim)
419     c1 = 2.0 / ((dim + 1.3) ** 2 + mu_eff)
420     c_mu = min(
421         1.0 - c1,
422         2.0 * (mu_eff - 2.0 + 1.0 / mu_eff) / ((dim + 2.0) ** 2 + mu_eff),
423     )
424
425     mean = random_feasible_layout(rng, n)
426     sigma = sigma0
427     C = np.eye(dim)
428     p_c = np.zeros(dim)
429     p_sigma = np.zeros(dim)
430     expected_norm = np.sqrt(dim) * (1.0 - 1.0 / (4.0 * dim) + 1.0 / (21.0
431                               * dim ** 2))
432
433     best_x = mean.copy()
434     best_score = f(best_x)
435
436     for generation in range(k_max):
437         C = 0.5 * (C + C.T)
438         eigenvalues, B = np.linalg.eigh(C)
439         eigenvalues = np.maximum(eigenvalues, 1e-12)
440         C_sqrt = B @ np.diag(np.sqrt(eigenvalues))
441         C_inv_sqrt = B @ np.diag(1.0 / np.sqrt(eigenvalues)) @ B.T
442
443         z = rng.standard_normal((m, dim))
444         y = z @ C_sqrt.T
445         samples = mean + sigma * y
446         samples = np.array([project_layout(sample, n) for sample in
447                               samples])
448         scores = np.array([f(sample) for sample in samples])
449
450         i_best = np.argmax(scores)
451         if scores[i_best] > best_score:
452             best_score = scores[i_best]
453             best_x = samples[i_best].copy()
454
455         elite_idx = np.argsort(scores)[-m_elite:][::-1]
456         elite_samples = samples[elite_idx]
457         old_mean = mean.copy()
```

```

455     mean = weights @ elite_samples
456     y_w = (mean - old_mean) / sigma
457
458     p_sigma = (1.0 - c_sigma) * p_sigma + np.sqrt(c_sigma * (2.0 -
459         c_sigma) * mu_eff) * (C_inv_sqrt @ y_w)
459     norm_p_sigma = np.linalg.norm(p_sigma)
460     h_sigma = norm_p_sigma / np.sqrt(1.0 - (1.0 - c_sigma) ** (2.0 * (
461         generation + 1))) < (
462         1.4 + 2.0 / (dim + 1.0)
462     ) * expected_norm
463     p_c = (1.0 - c_c) * p_c + h_sigma * np.sqrt(c_c * (2.0 - c_c) *
464         mu_eff) * y_w
464
465     elite_y = (elite_samples - old_mean) / sigma
466     rank_mu = sum(weights[i] * np.outer(elite_y[i], elite_y[i]) for i
467         in range(m_elite))
467     C = (
468         (1.0 - c1 - c_mu + (1.0 - h_sigma) * c1 * c_c * (2.0 - c_c)) *
469         C
470         + c1 * np.outer(p_c, p_c)
471         + c_mu * rank_mu
471     )
472     sigma *= np.exp((c_sigma / d_sigma) * (norm_p_sigma /
473         expected_norm - 1.0))
473
474     return best_x, best_score
474
475
476 def refine_top_layouts(
477     layouts,
478     n,
479     best_x,
480     best_score,
481     n_refines,
482     label,
483     slsqp_steps,
484     verbose=True,
485 ):
486     if not layouts or n_refines <= 0:
487         return best_x, best_score
487
488     unique_layouts = []
489     seen = set()
490     for x in layouts:
491         points = np.round(x.reshape(n, 2), 5)
492         points = points[np.lexsort((points[:, 1], points[:, 0]))]
493         key = tuple(points.reshape(-1))
494         if key not in seen:
495             seen.add(key)
496             unique_layouts.append(x)
496
497     scored = [(layout_score(x, n), x) for x in unique_layouts]
498     scored = [(score, x) for score, x in scored if score != -np.inf]
499     scored.sort(key=lambda item: item[0], reverse=True)
500
501
502

```

```

503     n_used = min(n_refines, len(scored))
504     for i, (_, x0) in enumerate(scored[:n_used]):
505         x, score = refine_layout_jax(x0, n, slsqp_steps)
506         if score > best_score:
507             best_x = x
508             best_score = score
509
510         if verbose:
511             log(f"n={n}: {label} refine {i+1}/{n_used}, best={best_score:.6f}")
512
513     return best_x, best_score
514
515 def optimize_layout(
516     n,
517     restarts=2,
518     k_max=15,
519     m=120,
520     m_elite=30,
521     seed=0,
522     boundary_starts=8,
523     greedy_starts=4,
524     final_refines=100,
525     slsqp_steps=100,
526     verbose=True,
527 ):
528     rng = np.random.default_rng(seed)
529     best_x = None
530     best_score = -np.inf
531     f = lambda x: layout_score(x, n)
532     candidate_layouts = []
533
534     if verbose:
535         log(f"n={n}: optimizing layout")
536
537     # Deterministic boundary candidates are cheap to score and often near
538     # active constraints.
539     feasible_boundary_starts = 0
540     projected_boundary_starts = 0
541     for i, phase in enumerate(np.linspace(0.0, 2.0 * np.pi,
542         boundary_starts, endpoint=False)):
543         x0 = outer_circle_layout(n, phase)
544         was_feasible = np.all(feasible_points(x0.reshape(n, 2)))
545         if not was_feasible:
546             x0 = project_layout(x0, n)
547             if not np.all(feasible_points(x0.reshape(n, 2))):
548                 continue
549             projected_boundary_starts += 1
550
551         if not np.all(feasible_points(x0.reshape(n, 2))):
552             continue
553
554     feasible_boundary_starts += 1
555     candidate_layouts.append(x0)

```

```

554
555     if verbose:
556         status = "feasible" if was_feasible else "projected"
557         log(
558             f"n={n}: boundary_start_{i+1}/{boundary_starts}, "
559             f"{status}, accepted={feasible_boundary_starts}",
560         )
561
562     if verbose:
563         log(
564             f"n={n}: boundary_starts_done "
565             f"({feasible_boundary_starts}/{boundary_starts} accepted, "
566             f"{projected_boundary_starts} projected)",
567         )
568
569     # Greedy max-min candidates cover outer, inner, exclusion, and
570     # interior samples.
571     greedy_layouts = greedy_candidate_layouts(rng, n, greedy_starts)
572     candidate_layouts.extend(greedy_layouts)
573     if verbose:
574         log(f"n={n}: greedy_candidate_starts_generated_{len(
575             greedy_layouts)}")
576
577     clique_layout, clique_score = discrete_clique_layout(rng, n, verbose=
578         verbose)
579     if clique_layout is not None:
580         candidate_layouts.append(clique_layout)
581         if clique_score > best_score:
582             best_x = clique_layout
583             best_score = clique_score
584         if verbose:
585             log(f"n={n}: discrete_clique_candidate_found, score={
586                 clique_score:.6f}")
587     elif verbose:
588         log(f"n={n}: no_discrete_clique_candidate_found")
589
590     # CMA-ES provides stochastic global search; SLSQP is applied only to
591     # top candidates.
592     for i in range(restarts):
593         if verbose:
594             log(f"n={n}: CMA-ES_restart_{i+1}/{restarts}_running")
595         x, score = cma_es(f, n, k_max, m, m_elite, rng)
596         candidate_layouts.append(x)
597         if score > best_score:
598             best_x = x
599             best_score = score
600         if verbose:
601             log(f"n={n}: CMA-ES_restart_{i+1}/{restarts}_done, best={
602                 best_score:.6f}")
603
604     best_x, best_score = refine_top_layouts(
605         candidate_layouts,
606         n,
607         best_x,

```

```

602     best_score,
603     final_refines,
604     "final_SLSQP",
605     slsqp_steps,
606     verbose=verbose,
607 )
608
609     return best_x, best_score
610
611 def efficiency(distance):
612     return 1.0 / (1.0 + 1.0 / distance)
613
614 def draw_constraints(ax):
615     ax.add_patch(Circle(ZONE_1, OUTER_RADIUS, fill=False, color="green",
616                        linewidth=2.0, label="Outer_boundary"))
617     ax.add_patch(Circle(ZONE_1, INNER_RADIUS, fill=True, color="gray",
618                        alpha=0.25, label="Inner_exclusion"))
619     ax.add_patch(Circle(ZONE_2, EXCLUSION_RADIUS, fill=True, color="red",
620                        alpha=0.25, label="High-beta_growth_exclusion"))
621     ax.add_patch(Circle(ZONE_3, EXCLUSION_RADIUS, fill=True, color="orange",
622                        alpha=0.25, label="Defensive_value_exclusion"))
623     ax.set_xlim(0.5, 9.5)
624     ax.set_ylim(0.5, 9.5)
625     ax.set_aspect("equal", adjustable="box")
626     ax.set_xlabel("Market_Beta_Score")
627     ax.set_ylabel("Value_HML_Score")
628     ax.grid(True, alpha=0.25)
629
630 def save_layout_plot(n, x, score, output_dir):
631     points = x.reshape(n, 2)
632     fig, ax = plt.subplots(figsize=(7, 7))
633     draw_constraints(ax)
634     ax.scatter(points[:, 0], points[:, 1], s=70, color="black", zorder=5,
635               label="Stocks")
636     for i, point in enumerate(points, start=1):
637         ax.annotate(str(i), point + np.array([0.08, 0.08]), fontsize=9)
638     ax.set_title(f"Optimized_Portfolio_Layout, n={n}, p*={score:.4f}")
639     ax.legend(loc="upper_right", fontsize=8)
640     fig.tight_layout()
641     fig.savefig(output_dir / f"layout_n{n}.png", dpi=200)
642     plt.close(fig)
643
644 def save_distance_plot(results, output_dir):
645     values = np.array(results)
646     fig, ax = plt.subplots(figsize=(7, 4.5))
647     ax.plot(values[:, 0], values[:, 1], marker="o", color="black")
648     ax.set_xlabel("Number_of_Stocks_n")
649     ax.set_ylabel("Optimized_Minimum_Pairwise_Distance_p*")
650     ax.set_title("Minimum_Pairwise_Distance_vs_Portfolio_Size")
651     ax.grid(True, alpha=0.3)
652     ax.set_xticks(values[:, 0].astype(int))
653     fig.tight_layout()
654     fig.savefig(output_dir / "minimum_distance_vs_n.png", dpi=200)
655     plt.close(fig)

```

```

651
652 def save_efficiency_csv(results, output_dir):
653     values = np.array([(n, score, efficiency(score)) for n, score in
654         results])
655     np.savetxt(
656         output_dir / "portfolio_efficiency_results.csv",
657         values,
658         delimiter=",",
659         fmt=["%d", "%.8f", "%.8f"],
660     )
661
662 def save_part1_outputs(results, layouts, output_dir):
663     output_dir.mkdir(parents=True, exist_ok=True)
664     for n in (3, 5, 7):
665         if n in layouts:
666             score = dict(results)[n]
667             save_layout_plot(n, layouts[n], score, output_dir)
668             save_distance_plot(results, output_dir)
669             save_efficiency_csv(results, output_dir)
670
671 def solve_task_1(output_csv=None, output_dir=None, optimizer_kwargs=None,
672     target_ns=None):
673     if output_csv is None:
674         output_csv = Path(__file__).with_name("portfolio_layout_results.
675             csv")
676     if output_dir is None:
677         output_dir = Path(__file__).with_name("part1_outputs")
678     output_dir = Path(output_dir)
679
680     existing = {}
681     if output_csv.exists():
682         data = np.loadtxt(output_csv, delimiter=",")
683         data = np.atleast_2d(data)
684         existing = {int(row[0]): float(row[1]) for row in data}
685
686     target_ns = range(2, 11) if target_ns is None else target_ns
687     results = [(n, existing[n]) for n in range(2, 11) if n in existing and
688         n not in target_ns]
689     layouts = {}
690     optimizer_kwargs = {} if optimizer_kwargs is None else
691         optimizer_kwargs
692
693     for n in target_ns:
694         log(f"starting_n={n}")
695         x, score = optimize_layout(n, seed=100 + n, **optimizer_kwargs)
696         results.append((n, score))
697         layouts[n] = x
698         log(f"finished_n={n}, p*={score:.6f}")
699
700     results.sort(key=lambda item: item[0])
701     np.savetxt(output_csv, np.array(results), delimiter=",", fmt=["%d", "
702         %.8f"])
703     log(f"saved_results_to_{output_csv}")
704     save_part1_outputs(results, layouts, output_dir)

```

```

699     log(f"saved_plots_and_efficiency_results_to_{output_dir}")
700     return results, layouts
701
702 def main():
703     LOG_FILE.write_text("")
704     optimizer_kwargs = dict(
705         restarts=3,
706         k_max=30,
707         m=350,
708         m_elite=50,
709         boundary_starts=720,
710         greedy_starts=500,
711         final_refines=100,
712         slsqp_steps=800,
713     )
714     solve_task_1(optimizer_kwargs=optimizer_kwargs, target_ns=(7, 9, 10))
715
716 if __name__ == "__main__":
717     main()

```

A.2 Gaussian Process

```

1  from pathlib import Path
2
3  import numpy as np
4
5  import matplotlib
6  matplotlib.use("Agg")
7  import matplotlib.pyplot as plt
8
9
10 LENGTH_SCALE = 4.0
11 NOISE_VARIANCE = 0.02 ** 2
12
13 def theoretical_efficiency(p):
14     return 1.0 / (1.0 + 1.0 / p)
15
16
17 def squared_exponential_kernel(x1, x2, length_scale=LENGTH_SCALE):
18     x1 = np.asarray(x1).reshape(-1, 1)
19     x2 = np.asarray(x2).reshape(1, -1)
20     return np.exp(-0.5 * ((x1 - x2) / length_scale) ** 2)
21
22 def fit_gp_predict(x_train, y_train, x_test):
23     # from book (noisy measurements):
24     #     X* -> x_test
25     #     X  -> x_train
26     #     y  -> y_train
27     # we want y_hat, computed from predicted mean and covariance (
28     #     posteriors)
29
30     mean_train = theoretical_efficiency(x_train)    # m(X)
31     mean_test  = theoretical_efficiency(x_test)    # m(X*)

```



```

31
32 k_pred = squared_exponential_kernel(x_test, x_test)           # K(X*,X*)
33 k_pred_train = squared_exponential_kernel(x_test, x_train)    #K(X*,X)
34 k_train = squared_exponential_kernel(x_train, x_train)         # K(X,X)
35
36 noisy_covariance = k_train + NOISE_VARIANCE * np.eye(len(x_train))
37
38 k_prod_inv = np.linalg.solve(noisy_covariance.T, k_pred_train.T).T
39
40 posterior_mean = mean_test + k_prod_inv @ (y_train - mean_train)
41 posterior_cov = k_pred - k_prod_inv @ k_pred_train.T
42 posterior_variance = np.maximum(np.diag(posterior_cov), np.finfo(float)
43                                 ).eps)
44
45 return posterior_mean, np.sqrt(posterior_variance)
46
47 def save_gp_plot(output_dir):
48     x_train = np.array([2.5, 2.5, 4.5, 4.5, 6.5, 6.5, 8.0, 8.0])
49     y_train = np.array([0.59, 0.62, 0.82, 0.85, 0.86, 0.88, 0.90, 0.91])
50     x_test = np.linspace(1.0, 10.0, 500)
51
52     mean, std = fit_gp_predict(x_train, y_train, x_test)
53     theory = theoretical_efficiency(x_test)
54
55     output_dir.mkdir(parents=True, exist_ok=True)
56
57     fig, ax = plt.subplots(figsize=(8, 5))
58     ax.fill_between(
59         x_test,
60         mean - 1.96 * std,
61         mean + 1.96 * std,
62         color="tab:blue",
63         alpha=0.18,
64         label="95% confidence region",
65     )
66     ax.plot(x_test, mean, color="tab:blue", linewidth=2.0, label="
67             Predicted GP posterior mean")
68     ax.plot(x_test, theory, color="black", linestyle="-", linewidth=2.0,
69             label="Theoretical efficiency over p")
70     ax.scatter(x_train, y_train, color="tab:red", s=20, zorder=5, label="
71             Simulation data")
72     ax.set_xlabel("Separation distance p")
73     ax.set_ylabel("Diversification efficiency")
74     ax.set_title("Gaussian Process Fit to Realized Efficiency")
75     ax.set_xlim(1.0, 10.0)
76     ax.set_ylim(0.45, 1.05)
77     ax.legend()
78     fig.tight_layout()
79     fig.savefig(output_dir / "gp_efficiency_fit.png", dpi=200)
80     plt.close(fig)
81
82     predictions = np.column_stack([x_test, mean, std, mean - 1.96 * std,
83                                     mean + 1.96 * std])
84     np.savetxt(

```

```
80         output_dir / "gp_efficiency_predictions.csv",
81         predictions,
82         delimiter=",",
83         header="p,mean,std,lower_95,upper_95",
84         comments="",
85     )
86
87
88 def main():
89     output_dir = Path(__file__).with_name("part2_outputs")
90     save_gp_plot(output_dir)
91     print(f"saved Part 2 GP outputs to {output_dir}")
92
93
94 if __name__ == "__main__":
95     main()
```